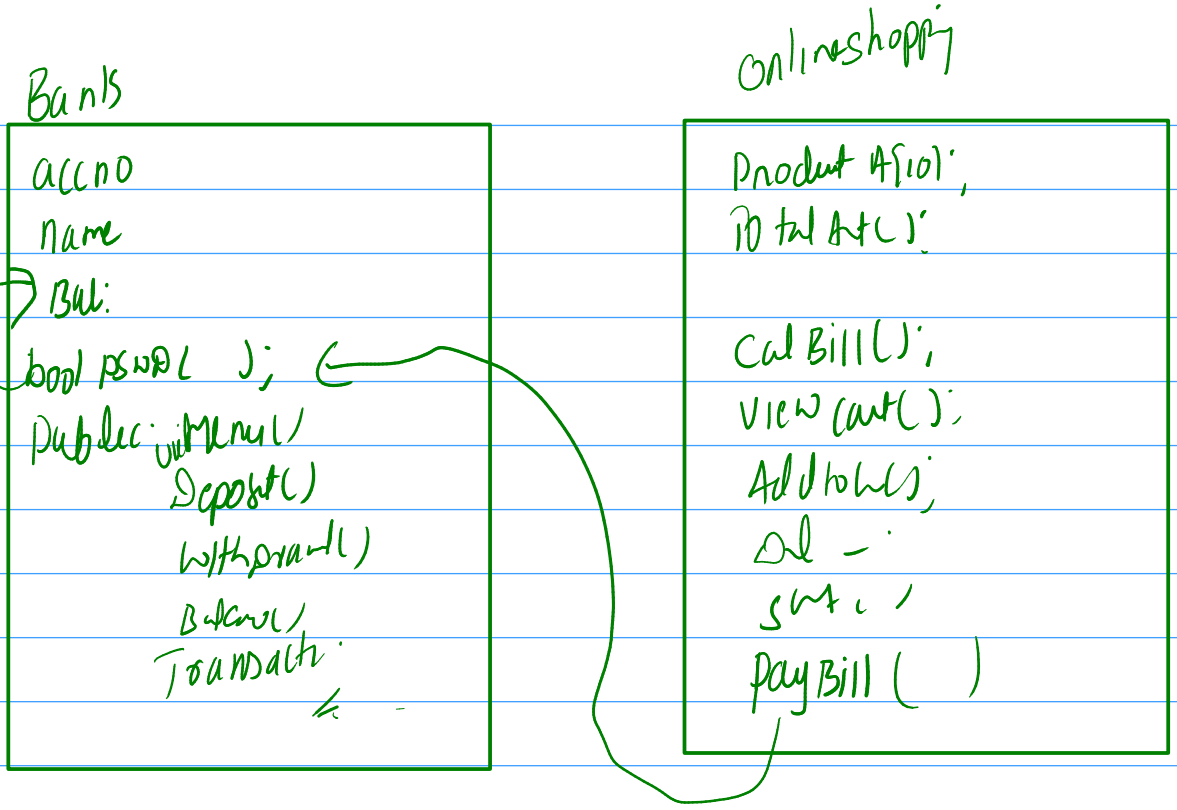
4/05




```
int x=10,y=20,z;
z=x+y;

int a[]={1,2,3},b[]={10,20,30},c[3];
c=a+b;
d=a-b;
char s1[]="abcd",s2[]="xyz",s3[100];

s3=s1+s2;
```

Add(a,b,c);
sub(a,b,c);
strcat(s1,s2);

Operator overloading

operator overloading is a compile time polymorphism , which is used to enhance the features or functionalites of the existing operators , so that we can apply on class objects.

Points to be noted :

1. while we are operator overloading we have to make sure that functionality , Priority and Associvity of the operator is not changed.
2. Using operator overloading we are not creating any new operators.

How do we overload operatorss?

1. using member functions of a class.
2. using friend functions of a class.

operator+

```
Complex & Add(Complex &e)
{
static Complex temp;
temp.real=real+e.real;
temp.img=img+e.img;
return temp;
}
```

c3 = c1.Add(c2);

c4 = c2.Sub(c3);

c3=c1+c2;

c4 = c2-c3;

operator –

```
Complex & Sub(Complex &e)
{ static Complex temp;
temp.real=real-e.real;
temp.img=img-e.img;
return temp;
}
```

As a program we use expression

ex :

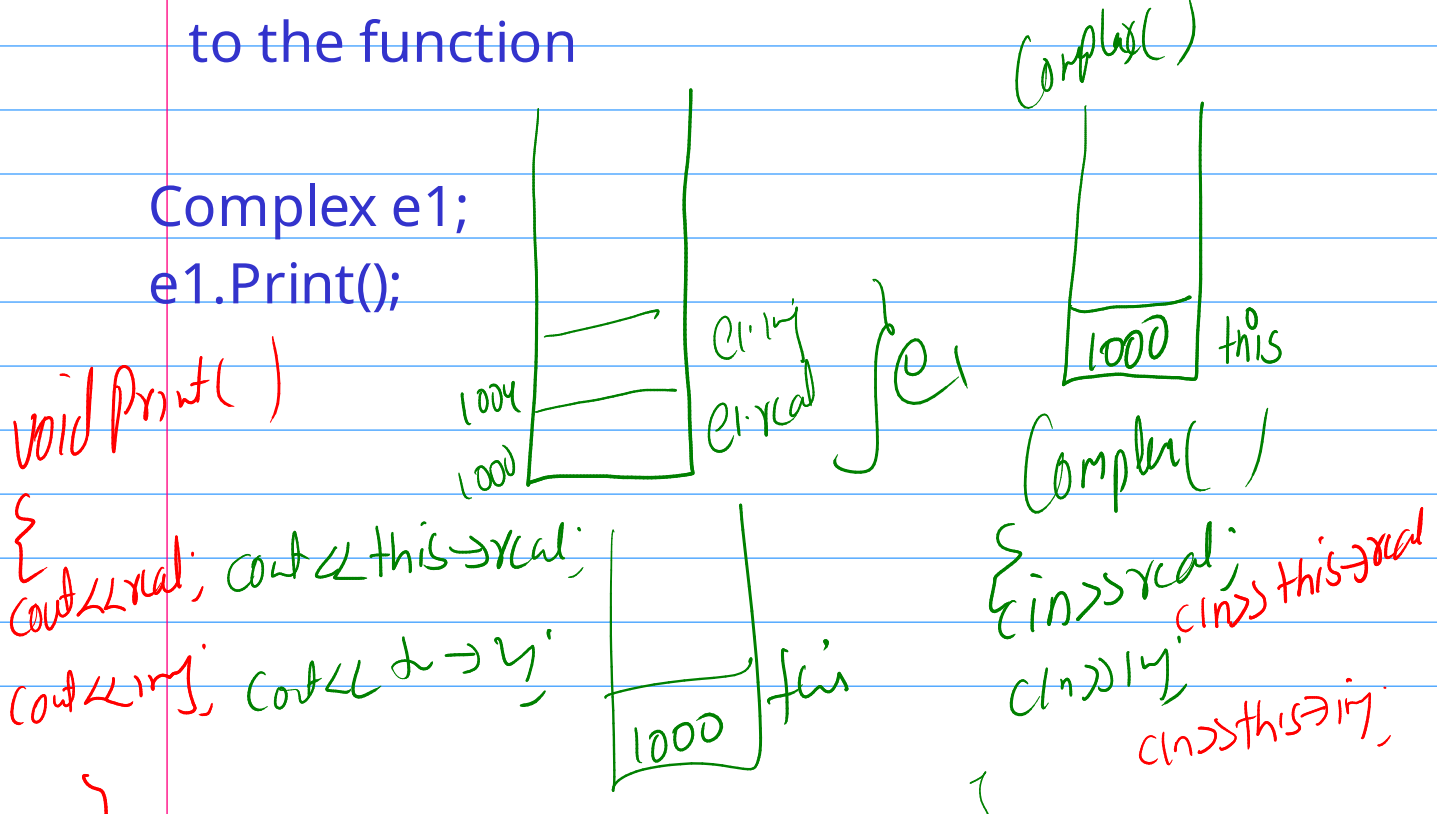
e3=e1+e2;

this expression is internally converted to

e3=e1.operator+(e2);

Compiler always makes left operand as calling object and passing right operand as parameter or input to the function

```
Complex e1;
e1.Print();
```



"this" is a hidden pointer supplied by the compiler in every member function of a class, except in static member function

"this pointer holds the base address of the calling object,
this pointer is ause to access the calling object or calling objects address inside the member functions

this ---> address of calling object
*this ----> calling object

Operators that cannot be overloaded:
1. scope resolution
2. sizeof
3. ternary
4. dot .
5. *.
6. typeid